Attack Surface Analysis

WebHarvester (Web Crawler Project)

CS493

Contents

1. Project & Team Information 3
2. Executive Summary 3
3. System Architecture Overview 4
4. Assets & Data Classification 6
5. Trust Boundaries 6
6. Attack Surface Components 7
   A. Network 7
   B. Software (Applications & APIs) 7
   C. Human (Users & Administrators) 7
   D. Execution environment (OS, containers, middleware) 7
   E. External dependencies (libraries, APIs, frameworks) 7
7. Threats & Attack Vectors (mapping) 8
8. Attack Surface Summary Table 9
9. Lessons Learned & Future Work 9
10. Revision Log 9

Instructor Checklist (for grading) 9

1. Project & Team Information

- **Project name:** PageHarvester
- **Team members:** O'Donovan Murphy II, Nikhil Patel, Omar Mountassir, Kevon Mair
- **Course / Instructor:** Dr. Zabihi
- **Date:** 4/26/26
- **Version / Revision:** v1.0

---

2. Executive Summary

PageHarvester is a program which autonomously parses a targeted Robots.txt file and downloads pages based on a search term entered by the user.

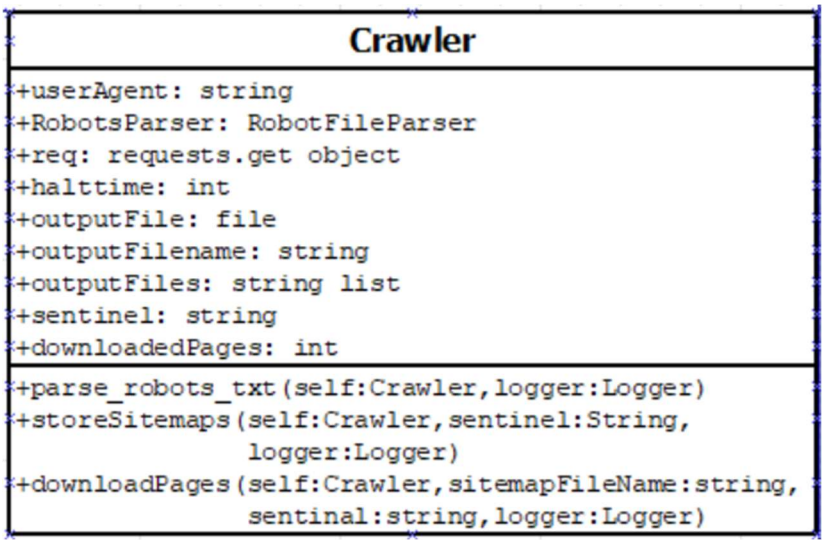
PageHarvester employs the following features:

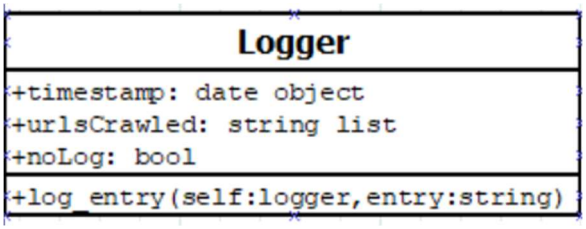
- Configurable, in code, delay between server requests. Defaulted to 5 seconds.
- Verification of URL access rights, based on Robots.txt, prior to autonomous access.
- Targeted crawling
- Storage of crawled results

---

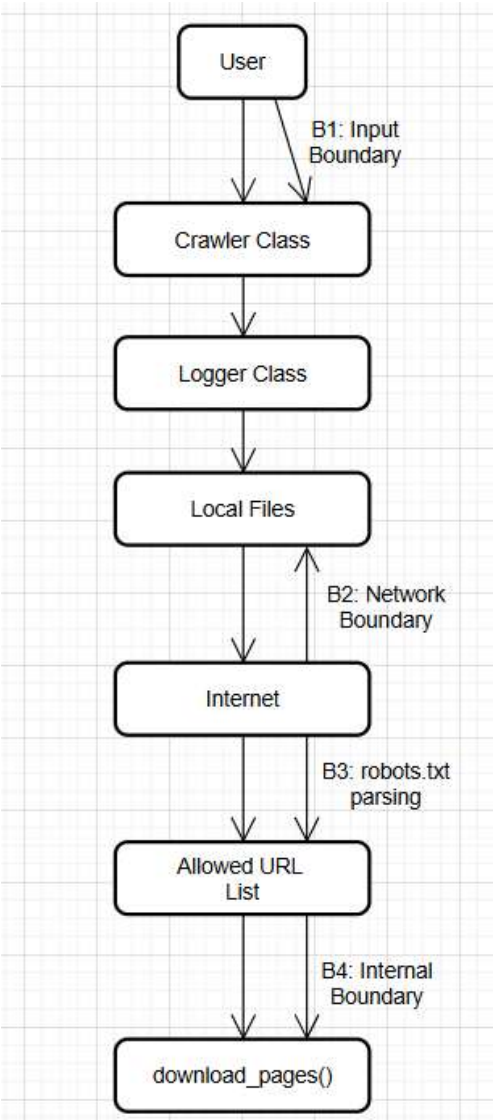
3. System Architecture Overview

- **UML Diagram:**
  The software is comprised of two classes. Crawler and Logger. Crawler handles most of the heavy lifting while the logger class sends basic status updates to a log file.





- **Trust Boundary Diagram:**



- **Primary components:**
  - **Requests**
  - **Pathlib**
  - **Datetime**
  - **Time**
  - **Re**
  - **Sys**
  - **Urllib.robotparser**

- **Data flows:**
  - Crawler class prompts user for a URL to crawl and a term to search for -> crawler.parse_robots.txt is called, if successful - > crawler.store_sitemaps is called.
    - Store sitemaps saves all xml files that are “allowable” and include the search term provided by the user. Crawler.outputFiles is incremented each time.
  - For each count of crawler.outputFiles crawler.download_pages is run. Storing html content of allowable and relevant pages.

---

4. Assets & Data Classification

Asset / Data	Owner	Description	Sensitivity (Low/Med/High)	Location
Raw HTML pages	Team/End User	Full downloaded pages	Low	~/_pages/”sentinel term”/
Stored XML Files	Team/End User	Sitemaps containing allowed URLs	low	~/_pages/”sentinel term” or ~/allowedSitemaps
Log file	Team/End User	Log file of operations	Low	~

---

5. Trust Boundaries

Boundary ID	From	To	Crossing mechanism	Assets exposed
B1	User	Input handler	Python input stream handling	Input stream, remaining processing.
B2	Internet	End User’s Machine	HTTP Requests	Raw HTML Files, IP address, log file.
B3	Robots.txt	Allowable URLs	RobotFileParser	Robots.txt, log file.
B4	Allowed URL list	DownloadPages	Internal source code.	Filesystem, log file.

---

6. Attack Surface Components

A. Network

- **External endpoints / ports:** HTTP & HTTPS Requests ports 80 and 443.
- **Outbound network:** Software makes outbound requests to servers of users choosing.
- **Protocols used:** HTTP(S)
- **Exposed services:** OS, networking drivers, ports

B. Software (Applications & APIs)

- **Crawler code / frameworks:** requests, pathlib, datetime, time, re, sys, urllib.robotparser
- **APIs & endpoints:** N/A
- **File handling:** Raw HTML and XML blobs are saved as ASCII files with the appropriate exention.
- **Third-party integrations:** N/A

C. Human (Users & Administrators)

- **Roles & privileges:** End User
- **Interfaces that accept input:** User input for robots.txt target and search term.
- **Authentication:** N/A
- **Human errors:** User could run multiple instances of the script and violate the 5 second delay.

D. Execution environment (OS, containers, middleware)

- **OS & versions:** OS Agnostic. Should run on any machine with a Python interpreter
- **Containers:** N/A
- **Middleware & runtime:** N/A

E. External dependencies (libraries, APIs, frameworks)

- **Libraries:** N/A
- **External services:** N/A

---

7. Threats & Attack Vectors (mapping)

Component	Element	Threat / Attack Vector	Potential Impact	Likelihood
Human	User Input	Someone could use the script to connect to a page that could be banned through their admin browser settings.	Violation of sysadmin policy	Low
Robot.txt parser	Parsing Robots.txt file	A malicious robots.txt could be hosted that is intentionally large and unable to be parsed.	Extreme memory usage and wasted CPU cycles	Low
Storage	Raw HTML & XML storage	Accidental storage of malicious or protected data.	Data leak, compliance violation, security risk	Medium
Environment	Dependencies	A vulnerability could exist in urllib.robotparser.	Exposure to outside attacks to end-user machines	Low-Medium
Network	IP blocking	Target sites block crawler IPs, or CAPTCHA challenges	Blacklisting from certain sites.	High

8. Attack Surface Summary Table

Exposure	Why it matters	Severity (1–5)	Remediation priority
Outbound fetches to untrusted hosts	Direct interaction with attacker-controlled servers	5	High
Raw HTML store that may malicious or protected data	Legal / privacy risk	2	Low
No vetting of 3rd party library uses with respect to security	Exposes the end-user to possible vulnerabilities.	5	High

---

9. Lessons Learned & Future Work

Parsing the robot.txt file from scratch is much more involved than originally anticipated. It also became obvious early on that many, if not most, are **very** restrictive when it comes to testing a webcrawler.

Future work would involve:

- Better error handling
- GUI
- Customizable pathing for storing crawled data
- MYSQL Database integration

---

10. Revision Log

- v1.0 — 2026 4/26/26 — Converted example to PageHarvester (crawler) project

---

Instructor Checklist (for grading)

- ☐ Diagram annotated with trust boundaries
- ☐ Inventory completed for network, software, human, execution environment, external dependencies
- ☐ At least 5 mapped threats with impacts and likelihood